SPARTA LOGISTICS · 01COMMIT10

전국 17개 허브, 주문부터 배송까지

업체 간(B2B) 주문 → 출발 허브 재고 차감 → 허브 간 이동 → 업체 배송.

서비스 5개로 나눈 물류 MSA

허브
지역 재고·물류 거점, 모든 허브가 직접 연결(P2P)

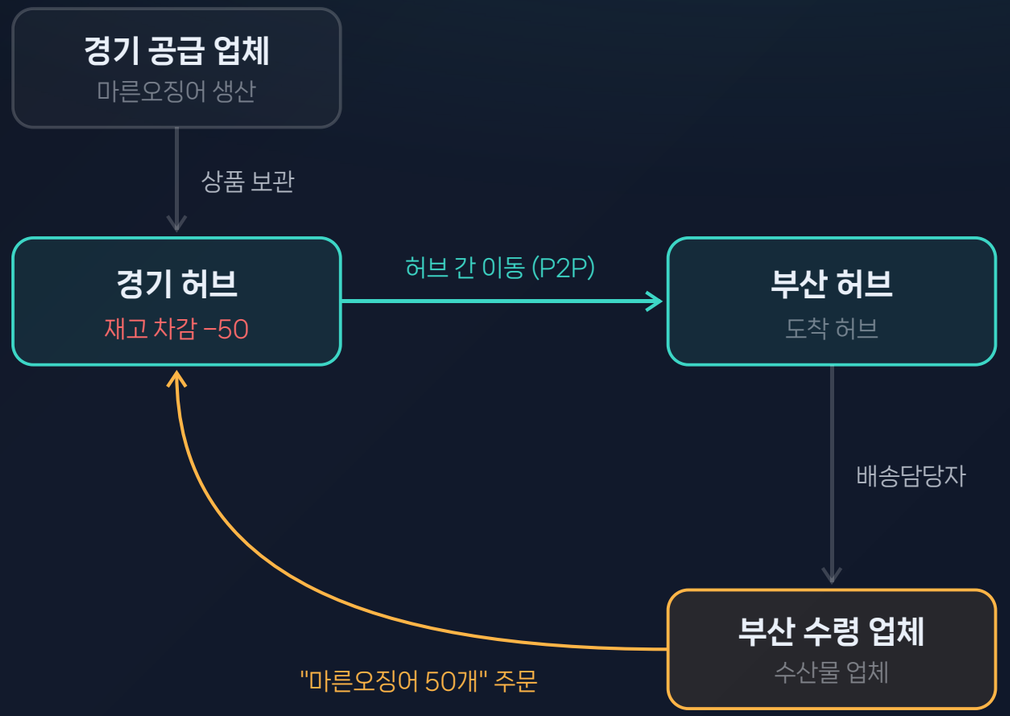
업체
특정 허브 소속, 생산업체이자 수령업체

주문
출발 허브 재고 차감 · 없으면 실패 · 취소 시 복원

배송
주문당 1건 + 구간(허브→허브→업체), 담당자 자동 배정

사용자
회원가입·로그인(JWT) · 권한 MASTER / USER · 배송담당자는 사용자 속성으로 지정

10/13 – 10/21 · user · hub · company · order · delivery



TEAM · 목표와 학습 주제

목표는 EDD — 이력서에 숫자로 쓸 수 있는 경험

팀 목표 · RESUME-DRIVEN DEVELOPMENT

"직접 측정하고 개선했다"고 한 줄로 쓸 수 있는 경험

학습 주제 · 고트래픽 대비 성능 개선

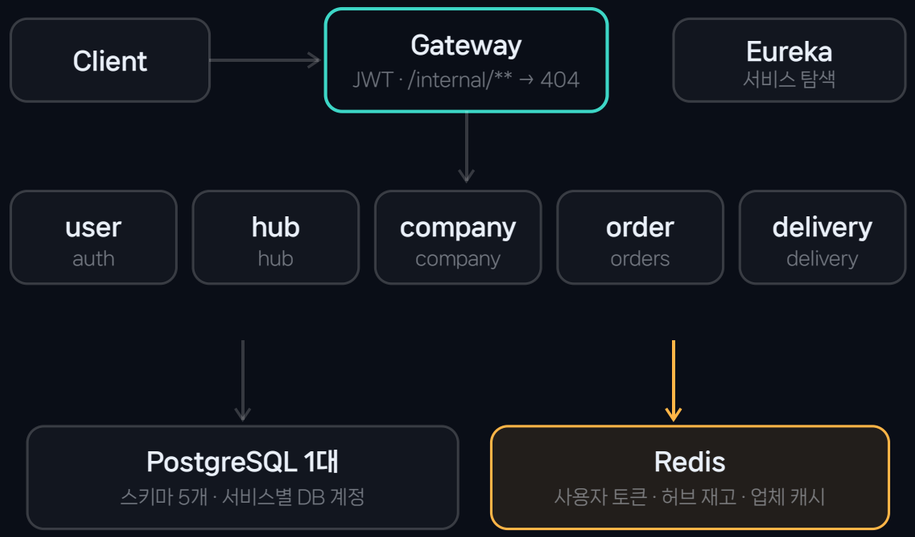
트래픽이 몰릴 때 어디서 병목이 일어나는지 체크하고 고친다 — 측정 → 개선 → 재측정

USER · GATEWAY	HUB	COMPANY · EUREKA	ORDER	DELIVERY
대진	이찬	권우	다울	영은
사용자 · 공통(Gateway) 성능 개선 설계	허브 · 경로 · 재고	업체 · 상품 공통(Eureka)	주문	배송

이력서 한 줄 예 — 인기 상품에 주문 800 TPS 집중, 425 → 707 TPS (+66%) · 행 락 대기 비중 79% → 0% · 재고 불일치 0건

ARCHITECTURE · 기술 선택 이유

스키마는 서비스마다, Redis는 사용자·허브·업체 세 곳에서



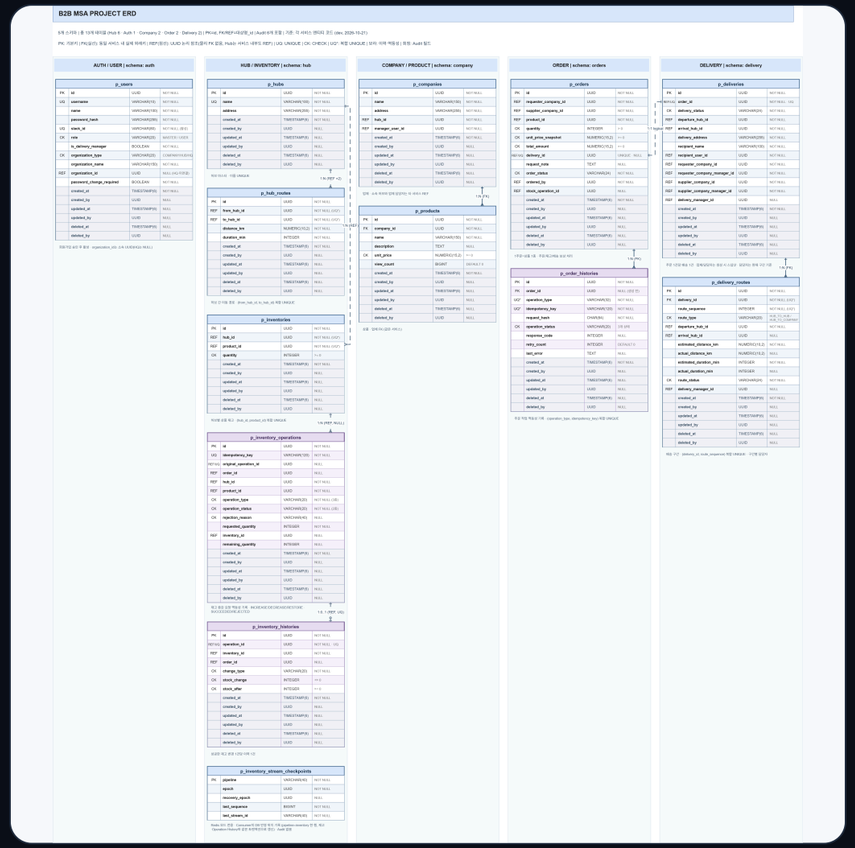
각 서비스 4계층 presentation → application → domain ← infrastructure

DB 분리	**DB 1대 + 스키마 5개**	일정 안에서 데이터 소유권 유지, 계정으로 남의 스키마 차단
서비스 통신	**OpenFeign (동기)**	Kafka 미도입 — 보상·유실 방지엔 강하지만 도메인마다 Producer·Consumer 필요 → 다음 목표
조회 캐시	**Caffeine + Redis**	서버 메모리(빠름) + 공유 캐시(일관) 2단
트랜잭션	**원격 호출은 트랜잭션 밖**	커넥션을 쥔 채 다른 서비스 응답을 기다리지 않게 — 주문·배송 생성에서 DB 작업만 짧게
재고 차감	**Redis Lua + Streams**	재고 차감 로직을 DB 행 락에서 Redis로 변경 — Lua 한 번으로 수량 확인·차감 후 바로 응답, 변경은 Streams 이벤트로 남겨 Consumer가 DB에 묶어 반영

근거 README 4장 · `settings.gradle` · `BlockedPathFilter` · `RedisTokenStateReader` · `hub-ms application.yaml`

ERD · API 설계

테이블 13개 · API 61개, 서비스끼리는 UUID로만 참조



전체 ERD · 클릭하면 확대 · 실선 = 같은 스키마 FK / 점선 = UUID 논리 참조

테이블	**13개** · 스키마 5개 (사용자 1 · 허브 6 · 업체 2 · 주문 2 · 배송 2)
API	**61개** · 외부 49 · 내부 12
공통	UUID PK · Audit 6컬럼 · Soft Delete
외부 API	`/api/v1/**`
내부 API	`/internal/v1/**` — Gateway에서 404
오류	`{code, message, errors}` 공통 형식
목록	공통 페이지 규칙 · `createdAt DESC`

설계 결정 2가지

- **멱등성** — 주문 이력 (`operation_type, idempotency_key`), 재고 작업 `idempotency_key`, 배송 `order_id` UNIQUE
- **배송담당자 = 사용자 속성** `is_delivery_manager` — 권한은 MASTER / USER 2단계 유지

근거 엔티티 `@Table` · 컨트롤러 매핑 (dev `a02b795`) · `docs/design/05 ERD.md` · `ErrorResponse` · `PagePolicy` · `User.isDeliveryManager`

어려웠던 점과 해결 · 학습 방식

블랙프라이데이 10시를 재현해 병목을 순서대로 고쳤다

1 **단순한 출발점**
조회 = DB만
주문 = 원격 호출까지 한 트랜잭션

2 **트래픽 재현**
k6 동시 요청

3 **병목 측정**
처리량 · p99 · 커넥션 풀

4 **해결**
원인 한 곳만

5 **재측정** ↻
다음 병목 찾기

10:00 · 장면 1

사람들이 상품을 찾는다

상품 조회 폭증 → 2단 캐시 · hot 모드 (6–7)

10:01 · 장면 2

한꺼번에 주문한다 — 병목이 옮겨 다님

① 주문 → ② 배송 → ③ 허브

트랜잭션 분리 ×2 · 재고 차감 Redis (8–10)

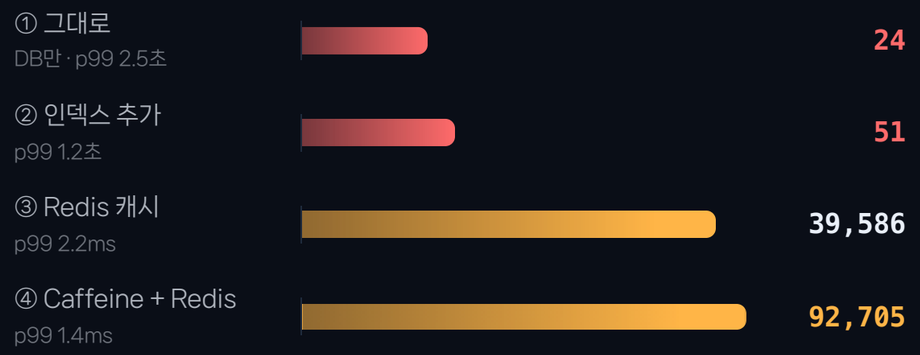
근거 k6 부하 테스트 · 노트북 1대 — 절대값보다 **비율과 병목 위치** 를 본다

장면 1 · 상품 조회가 몰린다

인덱스는 2배도 못 올렸고, 2단 캐시는 24 → 92,705 req/s

상품 **100만 건** · 동시 사용자 **50명** · 30초 연속 조회 (워밍업 10초 별도)

문제	TOP 10 **초당 24건 · p99 2.5초**, 검색 초당 19건 · p99 3.1초
원인	요청마다 DB가 **100만 행 전체 스캔** — 정렬 + 전체 건수 count, 상세는 요청마다 조회수 UPDATE
선택지	① 인덱스 ② Redis 캐시 ③ Caffeine(L1) + Redis(L2) 2단
판단	인덱스는 count가 여전히 전체 스캔, 1~2글자 검색은 트라이그램 불가 → 2배 이하. **DB에 가는 일 자체를 없앤다.** Redis만이면 왕복·역직렬화가 남아 L1 추가 (대가: 서버 여러 대면 L1 옛 값 최대 10초)
해결	L1 10초 + L2 상세 5분·검색 30초, 조회수는 Redis에 모아 5초마다 DB → **92,705 req/s · p99 1.4ms**



TOP 10 처리량(req/s) · 막대는 로그 눈금 · 검색은 19 → 20 → 19,540 → 53,833 (p99 3.1초 → 3.6초 → 2.5ms → 1.3ms)

근거 `docs/report/상품 조회 최적화 레포트.md` (10/19 측정) · 같은 구현 dev 반영 `53b3a5f` · `company-ms application.yaml` · `LayeredCache`

장면 1 · 동시 200명

캐시가 비는 순간 멈췄다 → 나눠 담아 최대 지연 19.5초 → 1.1초

문제 **캐시 스탬피드** – 200명이 검색할 때 캐시(30초)가 만료되면 **최대 19.5초 멈춤**

원인 같은 검색어를 기다리던 요청 수십 개가 **각자 DB로** 감 → 느린 검색 쿼리가 커넥션 풀을 다 잡음

선택지 ① `@Cacheable(sync=true)` ② Redis 잠금 writer ③ **같은 키는 한 요청만 DB** (키 단위 single-flight)

판단 ① Redis 기본 writer는 잠금이 없어 그대로 몰림 ② 캐시 이름 단위로 잠가 **다른 검색어까지 한 줄로** 섬 → ③ 다른 키는 서로 안 기다림

해결 같은 검색어가 동시에 비면 **첫 요청 1개만 DB**, 나머지는 그 결과를 받음 → 최대 지연 **19.5초 → 1.1초**, 처리량 7,861 → 43,690. TOP 10은 아예 안 비게 5초마다 덮어쓰기 → DB 전체 스캔 22 → 0번

API · 동시	S4 레이어드	S5 HOT
검색 · 200명	7,861 최대 19.5초	**43,690** 최대 1.1초 · 실패 0%
TOP 10 · 50명	95,656 전체 스캔 22번	96,663 **전체 스캔 0번**
상세 · 50명	33,865	24,817 **−27%**
검색 · 50명	50,356	34,214 **−32%**

대가: 50명 평소 처리량 상세 −27% · 검색 −32% – **몰리는 순간 무너지지 않는 쪽**을 택함

근거 `perf/product-query/results/p03/` (summary.md, `*-vu200.json`) · 같은 날 S4·S5 재측정 · dev PR #82

① order → ② delivery → ③ hub

주문 트랜잭션 분리 — 배송이 느려져도 60 → 581 TPS

문제 초당 주문 400부터 100씩 증가 → **500 TPS에서 order 커넥션 풀 고갈**, 600 TPS에서 p99 500ms 초과

원인 **주문 생성 로직이 하나의 트랜잭션으로 구성** — 주문 DB 저장이 같은 트랜잭션에 있어, 재고(hub)·배송(delivery) Feign 호출을 기다리는 동안에도 **DB 커넥션 풀을 점유**

선택지 ① 풀 키우기 (10 → 20) ② 원격 호출을 트랜잭션 밖으로 (구조 분리) ③ 둘 다

판단 풀만 키우면 점유 시간은 그대로라 배송이 0.3초 느려지면 `20 ÷ 0.333초 = 60` TPS로 무너짐 → **트랜잭션을 구조적으로 분리하고 풀도 10 → 20으로**(측정 설정). 600 TPS 커넥션 필요량 10.8 → **5.1개 (−53%)**, 배송 지연 시 60 → **581 TPS** (대가: 트랜잭션 3번, p50 +5ms)

해결 멱등키 선점 → 주문 저장 → 원격 호출 → 확정/실패 기록 · dev PR #78 Facade + TxService

60 → **581**

분리 전 · p99 56초 / 분리 후 · 9.7배

측정	분리 전	분리 후
배송 +300ms일 때 최대 처리	60 TPS	581 TPS
600 TPS 커넥션 필요량	10.8개	5.1개 −53%
커넥션 1회 점유 (배송 지연 시)	333ms	1.6ms
평소 p50 (트랜잭션 3번)	—	+5ms

↓ 진짜 가치는 속도보다 **격리** — 그런데 이번엔 **delivery 풀이 고갈**

미확인 perf 실험 브랜치 측정(TransactionTemplate 방식) · 원 보고서 레포 미수록 · dev 구현 PR #78 `OrderCreateFacade` + `OrderCreateTxService`

① order → ② delivery → ③ hub

배송도 같은 원인 — 원격 조회를 먼저, 커넥션 점유 −68%

문제	주문을 분리하자 병목이 옮겨 감 → 700 TPS에서 delivery 풀 고갈
원인	배송 생성 트랜잭션 안에서 업체·허브 경로·배송담당자를 원격 조회 — **주문과 같은 원인**
선택지	① 주문처럼 Facade + TxService로 구조 분리 ② 원격 조회만 트랜잭션 앞으로
판단	원인이 같으니 처방도 같다. 구조 분리는 변경 범위가 커서 보류 → ② **최소 변경**
해결	원격 조회를 먼저 끝내고 **배송·구간 저장만** 트랜잭션 (NOT_SUPPORTED 후 저장, `fa44b10`)

9.1ms → **2.9ms**

배송 1건 커넥션 점유 · 전 / 후 · delivery 풀 고갈 해소

↓ 그런데 → 이번엔 **hub 풀 고갈** (dev DB 방식: 600 TPS부터, 계단 전체 371초)

미확인 `perf 실험 브랜치 측정 · 원 보고서 레포 미수록 · dev 구현 fa44b10 DeliveryCommandService (NOT_SUPPORTED 후 저장)`

① order → ② delivery → ③ hub

재고 차감을 Redis Lua + Streams로 — 425 → 707 TPS

문제 인기 상품 하나에 800 TPS → DB 방식 **425 TPS · p99 7.4초**, hub DB 대기의 **79%가 행 락**

원인 성공한 주문마다 **재고 한 줄을 UPDATE** → 같은 행을 모두가 차례로 잠금 (SQL을 줄여도 444 → 503, 상품당 ~500이 상한)

선택지 ① Redis로 먼저 거르고 같은 요청에서 DB 즉시 반영 ② Redis에서 차감하고 DB는 Streams로 비동기 배치 반영

판단 ① DB가 기준으로 남지만 성공한 차감은 여전히 같은 행을 잡아 **행 락이 그대로** → ②, 대가(유실·지연)는 장치로 막는다

해결 `reserve.lua` 한 번: 멱등 확인 → 수량 검사 → 차감 → XADD → 바로 응답. Consumer가 최대 500건씩 한 트랜잭션 반영 후 XACK (PR #80·#81)

인기 상품 800 TPS	DB 방식	REDIS
최고 처리량	425 TPS	707 TPS (+66%)
p99	7.4초	4.2초
행 락 대기 비중	79%	0%
Redis ≠ DB · 음수 재고	–	0건 · 0
DB 반영 지연	–	평균 68~129ms · 최대 0.36초

유실·과부하 대비 (소스의 장치)

- `appendfsync always` + `noeviction` 아니면 요청 거부
- 미반영 **1만 건** 또는 **10초** 이상 → 차감 **503**
- DB 커밋 뒤에만 XACK · 실패하면 Pending 재처리

근거 dev 186ae33 재측정 10/21 (DH vs DH-pg) · docs/report/dev 주문 생성 부하 테스트 레포트.md · perf/order-create/results/ · dev PR #80·#81 · reserve.lua · InventoryStreamConsumer

단계별 측정 · 주문 생성 API POST /API/V1/ORDERS

고칠 때마다 다시 쟀다 — 버틸 수 있는 최대 500 → 800 TPS, 700 TPS p99 6.2초 → 0.16초

초당 주문 **400건**부터 **100건씩** 올려 **1,000건**까지, 계단마다 **60초** → 단계마다 주문 **29.4만 건** (9단계는 1,200건까지 43.2만 건) · 실패 0건

단계	바꾼 것	버틴 최대 (초당 건)	실제 처리 최대 (초당 건)	초당 700건일 때 P99	먼저 막힌 풀
1	기준 — 주문 1 트랜잭션 · 풀 10	500	597	6,231ms	order
3	order 트랜잭션 분리 (풀 10)	700	725	179ms	order·delivery
5	+ delivery 트랜잭션 분리 (풀 10)	700	727	79ms	order
7	+ 풀 20 + hub 차감 경량화	700~800	804~820	86~370ms	hub·order
9	+ 허브 경로 캐시 (풀 20)	**800**	**863**	158ms	order

배송이 0.3초 느려지면

트랜잭션 분리 전 60 TPS(p99 56초) → 분리 후 581 TPS · 풀만 키우면 평소엔 좋지만 여기서 무너짐

인기 상품 1개에 800 TPS

DB 방식 444 TPS(p99 7.3초) → 경량화 503 → dev Redis 재고 707 TPS(p99 4.2초), 행 락 대기 비중 0%

dev 그대로 재측정

600 TPS 버팀, 최대 689 TPS — company 상품 조회 1회 추가·경로 캐시 미반영으로 실험 최고(863)보다 한 계단 낮음

근거 판정: 버팀 = 처리량 98% 이상 + p99 500ms 이하 · 1~9단계 실험 브랜치 10/20~21 (7단계는 반복 측정 범위) · docs/report/주문 생성 부하 테스트 종합 보고서.md · perf/order-create/results/*-steps.csv

AI 활용 · 문제 → 방식 → 판단 → 검증

AI는 구현 도구가 아니라 검증 루프 — 기준은 테스트 커버리지

A · TDD 검증 루프

문제 Gateway 인가 우회·토큰 동시성 버그를 기능마다 빠짐없이

방식 계획 문서 → **실패 테스트 먼저** → 구현 → 뮤테이션으로 "테스트가 정말 잡나"

채택 테스트 + 뮤테이션까지 잡힌 구현

수정 뮤테이션 생존 → 케이스 보강 후 다시 Red

기각 **토큰 재발급 CAS** — 클라이언트 재시도 케이스에서 정상 세션 끊김

44	18/18	0
우회 재현 Red → Green	뮤테이션 검출	토큰 동시성 20회 → 0

B · 부하 테스트 분석

문제 커넥션 풀 고갈 원인을 숫자로 특정

방식 k6 시나리오·Hikari 지표 수집 스크립트·비교 레포트 초안

채택 트랜잭션 분리 → 주문당 커넥션 −53%

수정 "풀을 키우자" → 풀 ÷ 점유 시간으로 계산해 조정
Redis 재고 + 유실 대비(fsync always · 503)

기각 데이터 초기화 없이 잰 **470 TPS**

3%	9.7×	~800
계산 vs 실측 오차	배송 +300ms 60 → 581	초기화 후 재측정 TPS

근거 사례 B 출처: `docs/report/주문 생성 부하 테스트 종합 보고서.md` · `dev 주문 생성 부하 테스트 레포트.md` · `perf/order-create/` · 사례 A: gateway-edge·user-ms 테스트

잘 된 점 · 협업 문제와 해결

하나의 트랜잭션에서 출발해 하나씩 고쳐 버틴 최대 500 → 800 TPS

잘 된 점 1 · 점진적 개선 전략

단순한 구조(조회는 DB만, 주문은 한 트랜잭션)에서 출발해 **병목 하나 → 고침 → 재측정**을 반복

주문 버틴 최대 **500 → 800 TPS** · 배송 지연 시 **60 → 581 TPS** · 인기 상품 **425 → 707 TPS** · 검색 200명 최대 지연 **19.5초 → 1.1초**

잘 된 점 2 · 설정 스위치 + 측정을 레포에

재고·캐시 방식을 설정으로 바꿔 **같은 코드에서 전후 비교**, 문제 시 설정만 되돌림 · k6 스크립트·결과를 `perf/`에 남겨 **누구나 다시 잰다** (470 TPS 측정 오류도 재측정으로 발견)

협업 문제	해결
서비스마다 Feign 계약(필드·오류 코드) 어긋남	공통 모듈에 계약 먼저 — `AuthContract` 상수, 도메인별 `ErrorCodeRegistry`
로컬 포트 충돌 (Gateway 관리 포트 ↔ company-ms)	포트 표 확정 + 겹침 검사 테스트 `ActuatorPropertiesTest`
공통 모듈 변경이 남의 코드에 영향	[INSERT: 실제 공통 변경 규칙]
[INSERT: 일정·리뷰·브랜치]	[INSERT: 해결]

근거 수치: 종합 보고서 · dev 재측정 · `perf/product-query/results/p03/` · `ProductCacheMode` · `hub-ms application.yaml` · `common/.../AuthContract` · `gateway-edge/.../ActuatorPropertiesTest`

느낀 점 · 다음 목표

다음은 Kafka 비동기로 보상과 유실까지

Kafka 비동기 통신
보상 처리 · 메시지 유실 방지

옮겨 간 병목 마저
허브 경로 캐시 · company 상품 조회 캐시 (dev 600 → 실험 800 TPS) · Gateway 유입 제한

운영 준비
hub 기동 메모리 · Redis 기록 정리 · 여러 인스턴스 재측정

대진
1조분들 다 좋은 서비스 회사 들어가실길

이찬
아 여행가고싶다

권우
⭐대황진⭐ 킹이찬⭐ 퀸영은⭐ 갓다울⭐ 혼자서라면 이겨낼 수 없었을 시련들을 여러분들의 도움과 조언들 덕에 이겨낼 수 있었습니다. 여러분 모두 앞으로 나아갈 개발 여정에 꽃길만이 가득하길 바라겠습니다. 감사합니다!

다울
팀이라서 할 수 있었습니다. 소중한 우리 1조.

영은
설계는 중요하다.

Sparta Logistics · 01Commit10 · 질문 받겠습니다